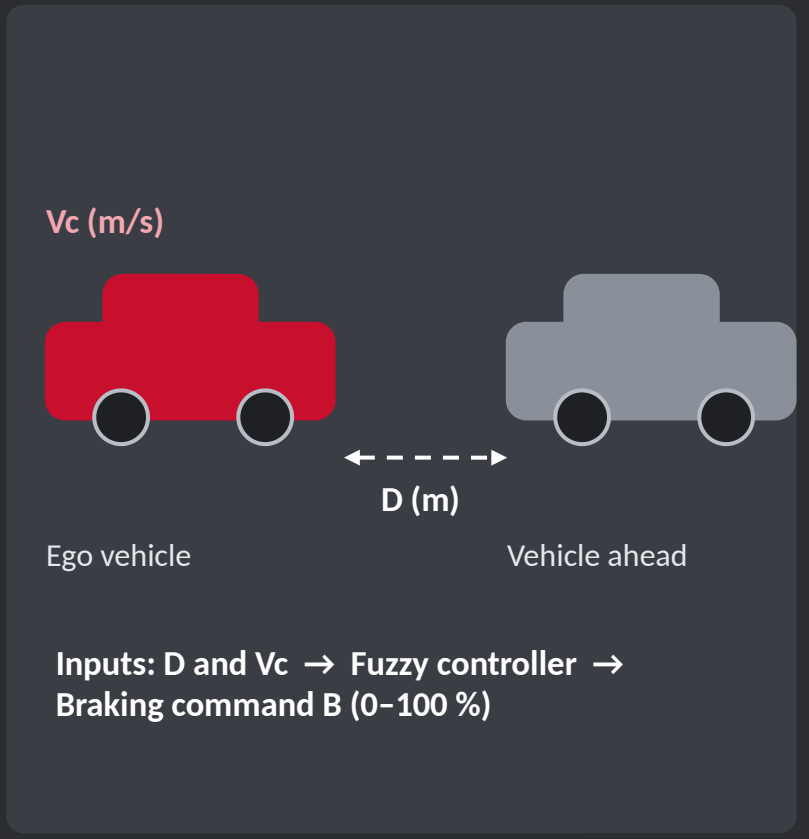
GTU | PROBLEM-BASED LEARNING (PBL) | FINAL PRESENTATION

Design and Development of a Fuzzy Logic-Based Automatic Braking System

Mamdani Fuzzy Inference System in Python (scikit-fuzzy)

Subject: Fuzzy Logic and Genetic Programming (BE05000461)
Department: Automobile Engineering
Institute: L.D. College of Engineering (LDCE), Ahmedabad
University: Gujarat Technological University (GTU)

Darshan Kantibhai Prajapati (Enrollment No: 250283102033)
Parth Sanjaybhai Parmar (Enrollment No: 250283102028)
Faculty Guide: [Name] | Academic Year: [Academic Year] | Date: [Date]



Illustrative schematic only – not project evidence

Problem Statement and Motivation

The problem

Drivers can react late to a closing obstacle because of distraction, fatigue or misjudged gaps. Braking decisions are vague: “near” and “fast” have no sharp boundary, which suits fuzzy logic better than crisp thresholds.

Why two inputs?

Distance alone ignores how quickly the gap is closing; speed alone ignores how much gap remains. A 20 m gap is relaxed at 2 m/s closing but urgent at 20 m/s. Together they approximate time-to-collision (TTC ≈ D / Vc).

Motivation and scope

Show how IF–THEN rules capture driver-like judgement and output a braking command (0–100 %). Scope: educational PBL software simulation. It does not claim to prevent collisions or replace production ADAS.

2

Inputs: D, Vc

1

Output: braking %

9

Fuzzy rules

Mamdani

Inference type

Objectives and System Architecture

1. Define 2 inputs and 1 output with ranges and units
2. Design 9 triangular membership functions (3 per variable)
3. Build a 9-rule Mamdani rule base (brief requires ≥ 6–8)
4. Hand-calculate one worked example (D = 20 m, Vc = 15 m/s)
5. Implement in Python with scikit-fuzzy; test ≥ 10 input pairs
6. Analyse behaviour, limitations and improvements

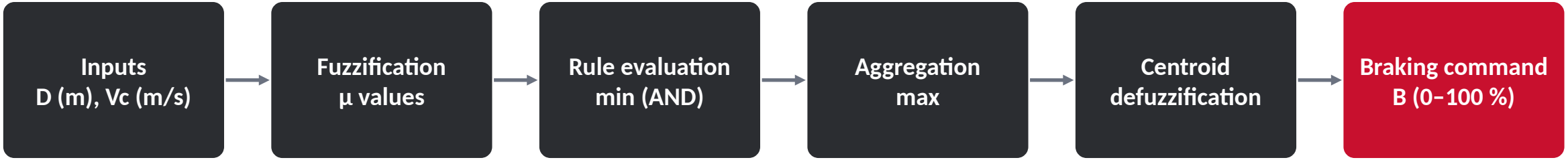


Fig. 1 – Mamdani FIS processing chain implemented in the simulation.

Simulation architecture only: no sensors, ECU, brake actuator or vehicle dynamics are modelled.

Variables and Membership Functions

Variable	Role	Unit	Universe	Terms and triangular MF [a, b, c]
Obstacle distance D	Input 1	m	0–100	Near [0,0,30] · Medium [10,40,70] · Far [50,100,100]
Closing speed Vc	Input 2	m/s	0–30	Slow [0,0,10] · Moderate [5,15,25] · Fast [20,30,30]
Braking command B	Output	%	0–100	Light [0,0,40] · Moderate [20,50,80] · Hard [60,100,100]

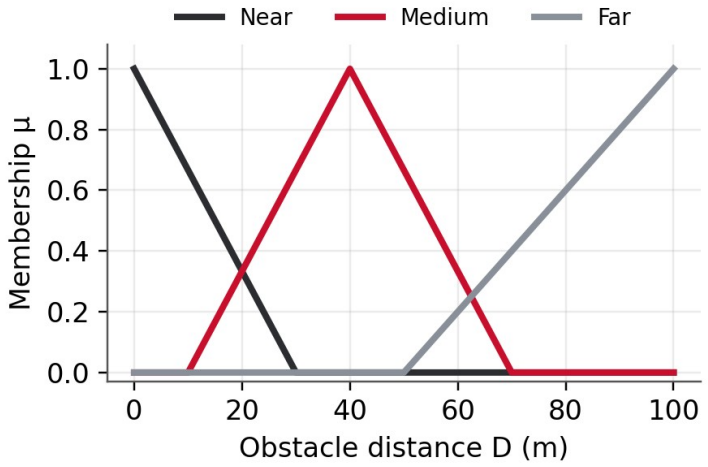


Fig. 2 – Distance MFs

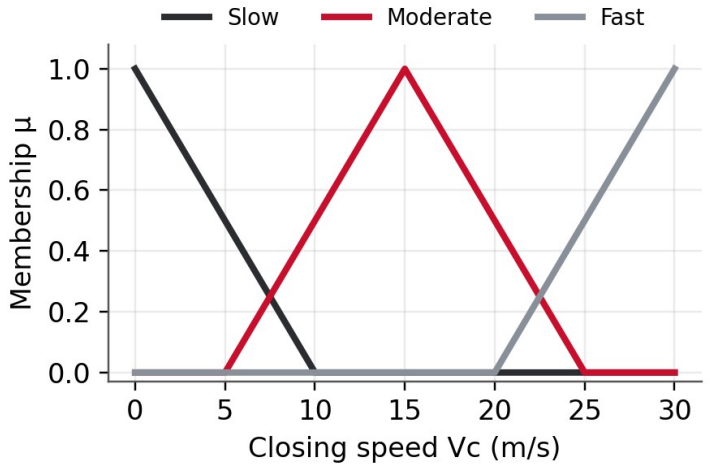


Fig. 3 – Closing-speed MFs

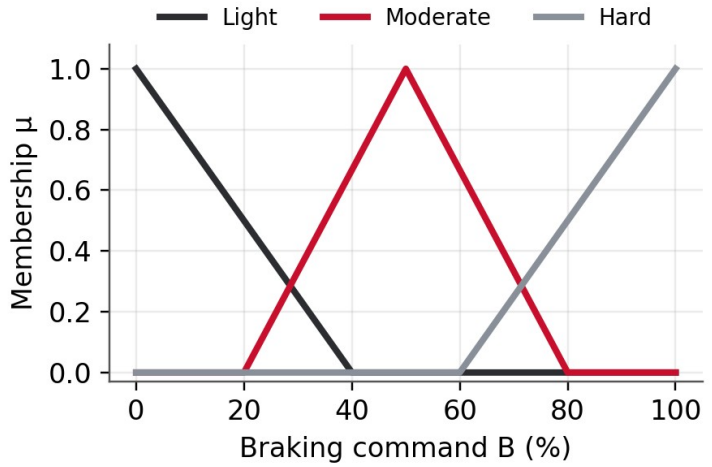


Fig. 4 – Braking-command MFs

Triangular MFs chosen for simplicity (Triangular / Trapezoidal / Gaussian allowed by brief). Edge-degenerate shapes suit a classroom demo; thresholds need engineering validation before real use.

Mamdani Fuzzy Inference Methodology

1

Fuzzification

Crisp D and Vc → membership degrees μ in each linguistic term

2

Rule evaluation

Antecedents combined with AND = min; output set clipped at α (min implication)

3

Aggregation

Clipped output sets of all rules combined with max

4

Defuzzification

Centroid of the aggregated set gives crisp braking command B*

Rule firing strength

$$\alpha_i = \min(\ \mu A_i(D), \mu B_i(Vc)\ )$$

Centroid defuzzification

$$B^* = \int b \cdot \mu agg(b)\, db \ \ / \ \int \mu agg(b)\, db$$

Evaluated numerically on a 1 % grid (exact piecewise-linear centroid, as in scikit-fuzzy).

Symbols

- α_i – firing strength of rule i
- μA_i, μB_i – membership of D and Vc in rule i's terms
- μagg(b) – aggregated output membership at braking level b
- B^* – crisp braking command (%)
- Operators: AND = min, implication = min, aggregation = max

Fuzzy Rule Base (9 Rules)

Distance ↓ / Vc →	Slow	Moderate	Fast
Near	R1 Moderate	R2 Hard	R3 Hard
Medium	R4 Light	R5 Moderate	R6 Hard
Far	R7 Light	R8 Light	R9 Moderate

Table 1 – Rule matrix: IF Distance is … AND Closing speed is … THEN Braking is …

Design logic

- Nearer obstacle and higher closing speed → stronger braking
- Far obstacle with low closing speed → light braking
- Near + Slow gives Moderate (not Hard): small closing rate leaves time to react
- Far + Fast gives Moderate: high speed warrants early action

A simplified demonstration rule base: it does not guarantee safe or optimal braking and must be validated against stopping-distance and vehicle-dynamics data before any real-world use.

Worked Example: D = 20 m, Vc = 15 m/s

Step	Calculation
Fuzzify D = 20	Near = (30−20)/(30−0) = 0.333 Medium = (20−10)/(40−10) = 0.333 Far = 0
Fuzzify Vc = 15	Slow = 0 · Moderate = 1 (peak) · Fast = 0
Rules fired	R2: Near ∧ Moderate → Hard α = min(0.333, 1) = 0.333 R5: Medium ∧ Moderate → Moderate α = min(0.333, 1) = 0.333 All other rules: α = 0
Aggregate	max of Hard clipped at 0.333 and Moderate clipped at 0.333
Defuzzify	**Centroid of aggregated set (executed model) = 62.4 %**

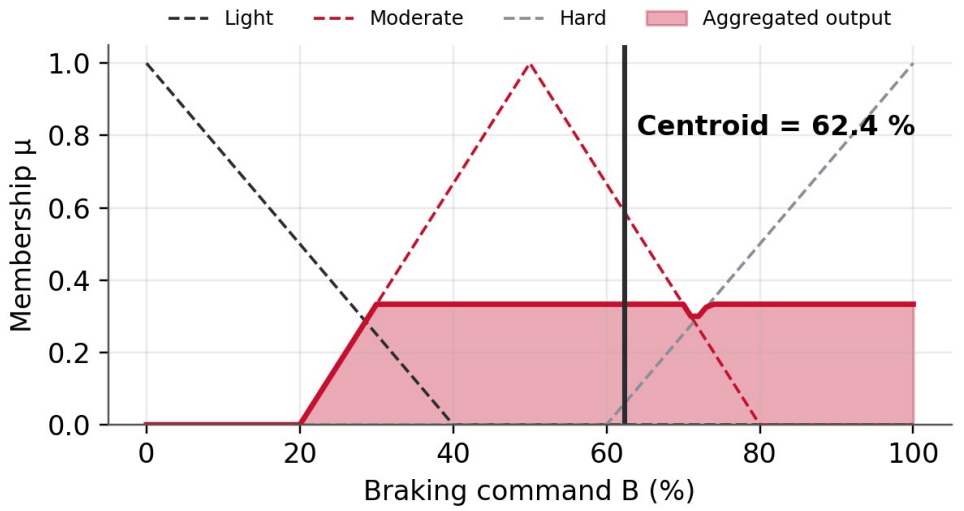


Fig. 5 – Aggregated output set and centroid from the executed model.

Why not a simple weighted average?
Averaging the label peaks (100 and 50) would give 75 %. The Mamdani centroid uses the full clipped shapes, so the true output is 62.4 %.

Python Implementation (scikit-fuzzy)

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

dist  = ctrl.Antecedent(np.arange(0,101,1),'distance')
speed = ctrl.Antecedent(np.arange(0,31,1),'closing_speed')
brake = ctrl.Consequent(np.arange(0,101,1),'brake')

dist['Near']   = fuzz.trimf(dist.universe,[0,0,30])
dist['Medium'] = fuzz.trimf(dist.universe,[10,40,70])
dist['Far']    = fuzz.trimf(dist.universe,[50,100,100])
speed['Slow']     = fuzz.trimf(speed.universe,[0,0,10])
speed['Moderate'] = fuzz.trimf(speed.universe,[5,15,25])
speed['Fast']     = fuzz.trimf(speed.universe,[20,30,30])
brake['Light']    = fuzz.trimf(brake.universe,[0,0,40])
brake['Moderate'] = fuzz.trimf(brake.universe,[20,50,80])
brake['Hard']     = fuzz.trimf(brake.universe,[60,100,100])
```

```
rules = [
 ctrl.Rule(dist['Near']   & speed['Slow'],
           brake['Moderate']),           # R1
 ctrl.Rule(dist['Near']   & speed['Moderate'],
           brake['Hard']),               # R2
 # ... R3 - R8 as in rule table ...
 ctrl.Rule(dist['Far']    & speed['Fast'],
           brake['Moderate']),           # R9
]
system = ctrl.ControlSystem(rules)
sim = ctrl.ControlSystemSimulation(system)

sim.input['distance']      = 20
sim.input['closing_speed'] = 15
sim.compute()
print(sim.output['brake'])   # centroid
```

Defaults in scikit-fuzzy: AND = min, aggregation = max, Consequent defuzzification = centroid. Full file: abs_fuzzy_skfuzzy.py

Verification note: scikit-fuzzy could not be installed in the build environment, so the numbers in this deck came from a NumPy re-implementation of the same operations (abs_numpy_check.py). Re-run the .py in your environment to confirm.

Simulation Interface and Demonstration

DESIGN CONCEPT – not a working simulator

Obstacle distance D (m)

20

Closing speed Vc (m/s)

15

Compute braking

Braking command: [shown after compute]

+ MF and output plots

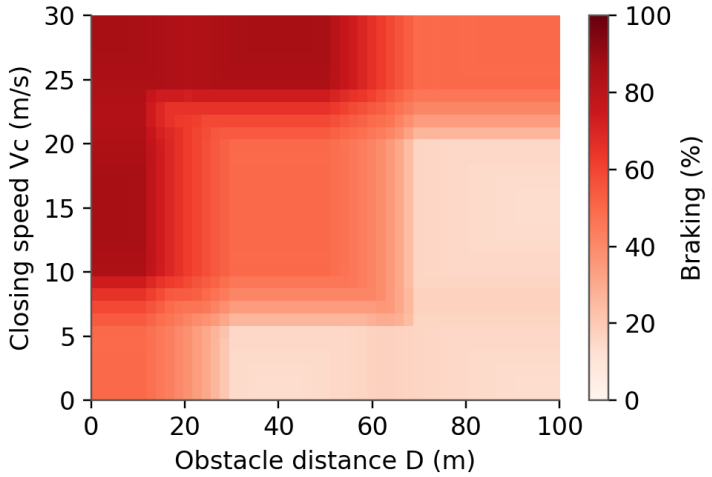


Fig. 6 – Braking surface (executed model)

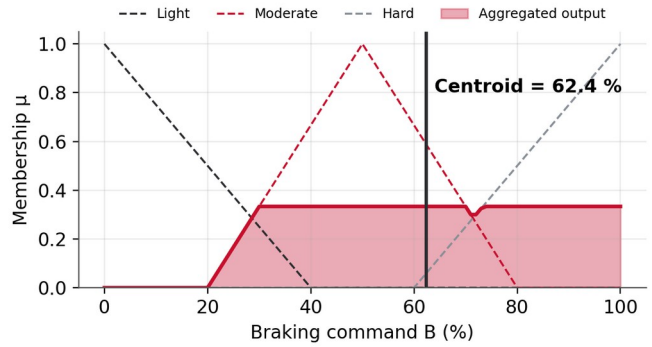


Fig. 7 – Output set, D=20, Vc=15

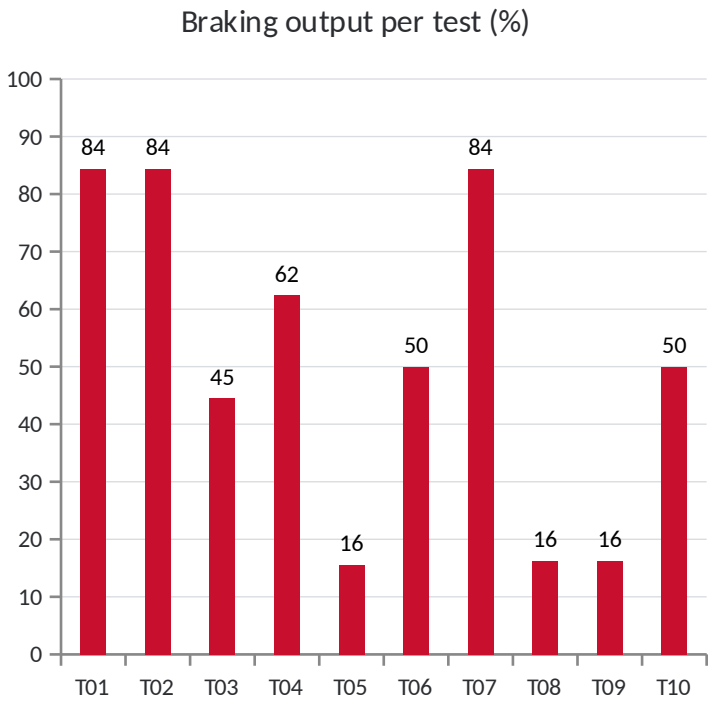
What the genuine plots show

- Output stays within 13.3–86.7 % over the whole input grid: centroids of clipped triangles never reach 0 % or 100 %.
- Braking rises smoothly toward short distance and high closing speed.

UI status (Phase 6): [pending – insert real Jupyter / Tkinter / Streamlit screenshot here]

Testing and Results (10 Input Combinations)

ID	D (m)	Vc (m/s)	Expected	Output (%)	Observation
T01	5	25	Hard	**84.4**	Near + Fast: Hard, clipped
T02	10	10	Hard	**84.4**	Near, partly Moderate: Hard
T03	15	5	Moderate	**44.5**	Near/Slow vs Medium/Slow mix
T04	20	15	Moderate–Hard	**62.4**	Worked example: Moderate–Hard blend
T05	40	5	Light	**15.6**	Medium + Slow: Light
T06	40	15	Moderate	**50.0**	Medium + Moderate: peak of Moderate
T07	40	25	Hard	**84.4**	Medium + Fast: Hard
T08	70	5	Light	**16.3**	Far + Slow: Light
T09	70	15	Light	**16.3**	Far + Moderate: Light (same as T08)
T10	90	28	Moderate	**50.0**	Far + Fast: Moderate



All outputs lie within 0–100 % and follow the rule base. T08 = T09 because Far gives α = 0.4 in both; the Light set is clipped identically. “Expected” is qualitative; outputs are computed (NumPy equivalent; see slide 8).

Analysis, Limitations and Future Improvements

Strengths

- Readable IF–THEN rules close to human reasoning
- Smooth transitions, no abrupt threshold jumps
- Easy to tune and extend with more inputs

Limitations observed

- Simplified 9-rule base; edge-degenerate MFs
- Output limited to 13.3–86.7 % (centroid effect)
- Far/Slow and Far/Moderate give identical output
- Sensitive to sensor noise in D and Vc

Not modelled

- Stopping distance, road friction, tyre grip
- Vehicle mass, actuator delay, brake dynamics
- Driver intent, road and weather conditions

Future work

- Add time-to-collision and acceleration inputs
- Validate against a physics-based vehicle model
- Compare with PID or tune MFs with a genetic algorithm
- Add noise-filtering and a real simulator UI

Proof-of-concept educational simulation – not ready for installation in a real vehicle.

Conclusion, Status and References

Conclusion

- A 9-rule Mamdani FIS maps distance and closing speed to a braking command that rises with urgency across 10 test cases.
- Worked example (D=20, Vc=15) gives 62.4 %, between Moderate and Hard as expected.
- Unvalidated: real stopping distances, vehicle dynamics, sensor noise, safety certification.

Deliverable	Status
Presentation	This file
Report	[Status to be updated]
Python source	abs_fuzzy_skfuzzy.py (run to confirm)
GitHub repository	[URL]
Short video	[Link]
LinkedIn post	[Link]

References

[1] L.D. College of Engineering, “PBL: Design and Development of a Fuzzy Controller for Real-World Engineering Problem,” BE05000461, course brief, 2026.

[2] scikit-fuzzy developers, “scikit-fuzzy documentation.” https://scikit-fuzzy.github.io/scikit-fuzzy/

[3] L. A. Zadeh, “Fuzzy sets,” Information and Control, vol. 8, no. 3, pp. 338–353, 1965.

[4] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” Int. J. Man-Machine Studies, vol. 7, no. 1, pp. 1–13, 1975.

[5] T. J. Ross, Fuzzy Logic with Engineering Applications, 4th ed. Chichester, U.K.: Wiley, 2016.